



El modelo avisa cuando cambia

El problema

La caja tiene una segunda ventana: el listado de pagos del día, con una tabla y un botón “Anular último”. Cada ventana tiene su controlador. `ControladorCaja` registra el pago en la `Caja` y después actualiza las dos ventanas. `ControladorListado` anula el último pago y actualiza la ventana que conoce, que es una sola:

```
// En ControladorCaja
private void registrar(Pago pago) {
    this.caja.registrar(pago);
    this.vista.mostrarTotal(this.caja.total()); // una línea por vista
    this.listado.actualizar();
}

// En ControladorListado
this.listado.getBotonAnular().addActionListener(evento -> {
    this.caja.anularUltimo();
    this.listado.actualizar(); // y VistaCaja? Este controlador no la conoce.
});
```

Después de tres pagos y un “Anular último”, la tabla tiene dos filas y el total sigue contando tres:

Caja
Monto:

Efectivo **Debito** **QR**

Total del día: \$2585.60

Pagos del día		
Medio	Monto	Acreditado
Efectivo	1500.00	1500.00
Debito	800.00	788.00
Anular ultimo		

Sin observador. El listado se actualizó, el total de la caja quedó viejo.

El estado está en la `caja`, pero cada ventana se actualiza solo cuando el controlador que hizo el cambio la llama. Con una tercera ventana hay que modificar los dos controladores. La salida: la `caja` llama a las vistas. Cada vista se registra en la `caja` como observadora, y la `caja`, después de cada cambio, llama a un método de cada una.

Observador

La `caja` define lo que necesita de una vista: un método para llamarla después de cada cambio. Es una interfaz del paquete `modelo`, igual que `Notificador`:



```
public interface ObservadorCaja {  
    void cajaCambio(Caja caja);  
}
```

La caja guarda una lista de observadores, acepta nuevos con `agregarObservador()`, y cada método que cambia el estado termina llamando a `notificar()`:

```
public class Caja {  
    private final List<Pago> pagos;  
    private final List<ObservadorCaja> observadores;  
    // ...  
    public void agregarObservador(ObservadorCaja observador) {  
        this.observadores.add(observador);  
    }  
  
    public boolean registrar(Pago pago) {  
        this.pagos.add(pago);  
        this.notificar();  
        return true;  
    }  
  
    public boolean anularUltimo() {  
        this.pagos.remove(this.pagos.size() - 1);  
        this.notificar();  
        return true;  
    }  
  
    private void notificar() {  
        for (ObservadorCaja observador : this.observadores) {  
            observador.cajaCambio(this);  
        }  
    }  
}
```

La Caja no importa `javax.swing` y no tiene una referencia a ninguna vista. Tiene una lista de `ObservadorCaja`, y lo único que hace con cada uno es llamar a `cajaCambio()`. Quiénes son y qué hacen con el aviso es asunto de cada observador.

El modelo define la interfaz del observador y llama a cada observador después de cada cambio.

Las vistas observan

`VistaCaja` implementa `ObservadorCaja`. En `cajaCambio()` lee el total de la caja que recibe y lo muestra:

```
public class VistaCaja implements ObservadorCaja {  
    // ...  
    @Override  
    public void cajaCambio(Caja caja) {  
        this.mostrarTotal(caja.total());  
    }  
}
```



Los controladores dejan de actualizar vistas. registrar() queda en una línea, this.caja.registrar(pago), y el receptor del botón de anular en otra, this.caja.anularUltimo(). ControladorCaja ya no recibe el listado por constructor. El registro de las observadoras va en Programa, junto con los otros new:

```
Caja caja = new Caja(new ServicioCorreo("smtp.lapampa.com", 587));
VistaCaja vista = new VistaCaja();
VistaListado listado = new VistaListado(caja);
caja.agregarObservador(vista);
caja.agregarObservador(listado);
new ControladorCaja(vista, caja);
new ControladorListado(listado, caja);
```

Caja

Monto:

Efectivo Debito QR

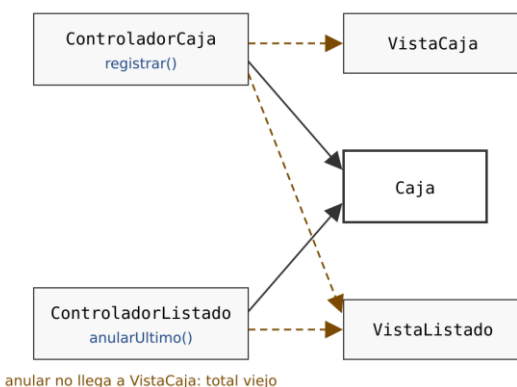
Total del dia: \$2288.00

Pagos del día		
Medio	Monto	Acreditado
Efectivo	1500.00	1500.00
Debito	800.00	788.00
Anular ultimo		

Con observador. Después de anular, las dos ventanas muestran lo mismo.

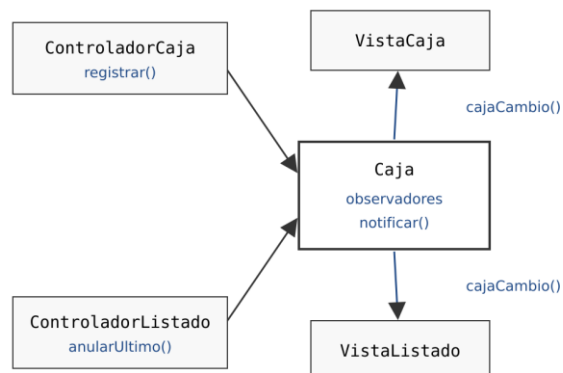
Los controladores escriben en la caja. Las vistas leen de la caja, cuando la caja avisa. Una tercera ventana es una clase que implementa ObservadorCaja y una línea en Programa; los controladores no cambian.

Antes: cada controlador actualiza sus vistas



anular no llega a VistaCaja: total viejo

Después: la Caja avisa a los observadores



Los controladores escriben en la Caja. Las vistas leen de la Caja, cuando la Caja avisa.

Antes y después. Las flechas punteadas son las actualizaciones a mano; las azules, los avisos de la Caja.

La vista lee del modelo cuando el modelo avisa. Escribir en el modelo sigue siendo del controlador.



JTable y TableModel

La tabla del listado es una JTable. Una JTable no guarda datos: dibuja lo que responde un TableModel, un objeto con cuatro métodos: cuántas filas, cuántas columnas, cómo se llama cada columna y qué va en cada celda. AbstractTableModel trae todo lo demás resuelto, y alcanza con heredar de ella:

```
public class ModeloTablaPagos extends AbstractTableModel {
    private static final String[] COLUMNAS = {"Medio", "Monto", "Acreditado"};
    private final Caja caja;
    // ...
    public int getRowCount() { return this.caja.getPagos().size(); }
    public int getColumnCount() { return COLUMNAS.length; }
    public String getColumnName(int columna) { return COLUMNAS[columna]; }

    public Object getValueAt(int fila, int columna) {
        Pago pago = this.caja.getPagos().get(fila);
        switch (columna) {
            case 0: return pago.medio();
            case 1: return String.format("%.2f", pago.getMonto());
            case 2: return String.format("%.2f", pago.montoAcreditado());
            default: return "";
        }
    }
}
```

Los datos siguen en la Caja: el TableModel los lee cada vez que la tabla los pide. La tabla se crea con `new JTable(modeloTabla)`, adentro de un `JScrollPane`, que dibuja el encabezado y la barra de desplazamiento.

VistaListado también implementa `ObservadorCaja`, y en `cajaCambio()` llama a `modeloTabla.fireTableDataChanged()`. Ese método avisa a la JTable que los datos cambiaron, y la tabla vuelve a pedir filas y celdas. Es el mismo patrón, adentro de la biblioteca: la JTable es observadora de su TableModel, y `fireTableDataChanged()` es su `notificar()`.

Una JTable dibuja lo que responde su TableModel, y se redibuja cuando el TableModel avisa.

En la biblioteca

El patrón se llama Observer, y aparece en Swing con distintos nombres. Un JButton es un objeto observado, `addActionListener()` registra observadores, y `actionPerformed()` es el aviso. Un TableModel avisa a sus `TableModelListener`, y la JTable es uno. Para un modelo propio, Java trae `PropertyChangeSupport`: `addPropertyChangeListener()` registra, `firePropertyChange("total", viejo, nuevo)` avisa, y el observador implementa `propertyChange(PropertyChangeEvent evento)`. Es la misma lista y el mismo for que Caja, con nombres genéricos. Para un modelo con dos o tres cambios distintos, una interfaz propia como `ObservadorCaja` queda más clara.

Observer: el observado tiene una lista de observadores y los llama cuando cambia. ActionListener, TableModelListener y PropertyChangeListener son el mismo patrón.



Qué se ganó

Los controladores no conocen a las vistas para actualizarlas: escriben en la `Caja` y nada más. Las ventanas muestran siempre lo mismo, porque las dos reciben el mismo aviso después de cada cambio. Y una ventana nueva entra con una clase y una línea en `Programa`.

Bibliografía sugerida para profundizar

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- El patrón “Observer”, en el capítulo 5, “Behavioral Patterns”. Es la descripción original, con el ejemplo de un mismo dato mostrado en una tabla y en un gráfico.

Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2.^a ed.). O’Reilly.

- Capítulo 2, “The Observer Pattern”. La explicación más gradual, con una versión propia del patrón y después la de la biblioteca.

Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.

- Capítulo 24, “Observer: Backing into a Pattern”. El patrón aparece al refactorizar un reloj digital.

Oracle. *The Java Tutorials*, “How to Use Tables” y “How to Write a Property Change Listener”.

- La referencia oficial de `JTable` y `TableModel`, con el modelo de tabla mínimo, y la de `PropertyChangeSupport`.